```
0000 WITH LALRIDL_NODE_ATTR_CLASS_NAMES;¶USE  LALRIDL_NODE_ATTR_CLASS_NAMES;¶--|-------------------------------------------------------------------------------
     ----------------------¶--|¬¬IDL¬¬¬¬¬¬¬¬¬--| VERSION SPECIALISEE POUR LE TRAITEMENT DE GRAMMAIRE¶--|-------------------------------------------------------------------------------
     ----------------------------------------------¶PACKAGE IDL IS¶¶¬--| CREATION/OUVERTURE FERMETURE DE FICHIER .LAR (FICHIER ARBRE)¶¶  PROCEDURE CREATE_ID
     L_TREE_FILE¬( PAGE_FILE_NAME :STRING );¬¬¬¬¬--| CREE UN FICHIER¬"PAGE_FILE_NAME.LAR"¶  PROCEDURE OPEN_IDL_TREE_FILE¬( PAGE_FILE_NAME :STRING );¬¬¬¬¬--|
     OUVRE¬UN FICHIER "PAGE_FILE_NAME.LAR" DEJ CRE¶  PROCEDURE CLOSE_IDL_TREE_FILE;¬¬¬¬¬¬¬¬--| FERMETURE DU FICHIER ".LAR"¶¶¬--| UTILITAIRES POUR GRAMMAIRE
     LALR¶¶       PROCEDURE READ_GRMR¬( NOM_TEXTE :STRING¬);¶       PROCEDURE OPTR_GRMR¬( NOM_TEXTE :STRING¬);¶       PROCEDURE INIT_GRMR¬( NOM_TEXTE :STRI
     NG¬);¶       PROCEDURE STAT_GRMR¬( NOM_TEXTE :STRING¬);¶       PROCEDURE LALR_GRMR¬( NOM_TEXTE :STRING¬);¶       PROCEDURE CHECK_GRMR¬( NOM_TEXTE :STR
     ING¬);¶       PROCEDURE PRINT_STAT¬( NOM_TEXTE :STRING¬);¶       PROCEDURE LOAD_GRMR¬( NOM_TEXTE :STRING¬);¶¶¶¬--| DISPOSITIF D'ACCES A UN ARBRE REPES
     ENTANT UNE¬DESCRIPTION DE GRAMMAIRE¶¶  TYPE SHORT¬¬¬IS RANGE -32_768 ..¬32767;¬FOR SHORT'SIZE    USE 16;¶  TYPE POSITIVE_SHORT¬¬IS RANGE 0 .. 32767;¬F
     OR POSITIVE_SHORT'SIZE   USE¬15;¶  TYPE PAGE_IDX¬¬¬IS RANGE 0 .. 16#7FFF#;¬FOR PAGE_IDX'SIZE USE 15;¶  TYPE LINE_IDX¬¬¬IS RANGE 0 .. 127;¬¬FOR LINE_ID
0010 X'SIZE USE 7;¶  SUBTYPE¬ATTR_NBR¬¬¬IS LINE_IDX;¶  TYPE LINE_NBR¬¬¬IS RANGE 0 .. 128;¶¶  TYPE SRCCOL_IDX¬¬¬IS RANGE 0 .. 255;¬¬FOR SRCCOL_IDX'SIZE¬USE
     8;¶¶  TYPE VPTR_TYPE¬¬¬IS (P, S,¬L, HI);¬¬¬¬¬¬--| PTR NOEUD, SOURCE_POS, LIST, HEADER/INTEGER¶  TYPE TREE (PT : VPTR_TYPE := P)¬IS¬¬¬¬¬¬--| PAR DEFAUt
     POINTEUR DE NOEUD¶¬¬RECORD¶¬¬  CASE PT¬IS¶¬¬  WHEN P | L =>¬¬¬¬¬¬¬¬--| POINTEUR NORMAL¬DE NOEUD OU ATTRIBUT LISTE¶¬¬    TY¬¬: NODE_NAME;¬¬¬¬¬¬--| TYPE D
     E NOEUD¶¬¬    PG¬¬: PAGE_IDX;¬¬¬¬¬¬--| REFERENCE DE PAGE VIRTUELLE¶¬¬    LN¬¬: LINE_IDX;¬¬¬¬¬¬--| DECALAGE DANS UNE PAGE VIRTUELLE¶¬¬  WHEN S =>¬¬¬¬¬¬¬¬
     -| POINTEUR DE SOURCE_LINE AVEC COLONNE SOURCE EN PLACE DU¬TYPE¶¬¬    COL¬¬: SRCCOL_IDX;¬¬¬¬¬¬--| NUMERO DE COLONNE DANS LE¬TEXTE SOURCE¶¬¬    SPG¬¬: P
     AGE_IDX;¬¬¬¬¬¬--| REFERENCE DE PAGE VIRTUELLE¶¬¬    SLN¬¬: LINE_IDX;¬¬¬¬¬¬--| DECALAGE DANS UNE PAGE VIRTUELLE¶¬¬  WHEN HI¬=>¬¬¬¬¬¬¬¬--| HEADER DE NOEUD¬
     OU INTEGER (SHORT) CODE PAR VALEUR ABSOLUE ET INDICATEUR DE¬COMPLEMENT A DEUX¶¬¬    NOTY¬¬: NODE_NAME;¬¬¬¬¬¬--| TYPE DE NOEUD¶¬¬    ABSS¬¬: POSITIVE_SH
     ORT;¬¬¬¬¬¬--| VALEUR ABSOLUE D UN SHORT¬POUR UNE VALEUR ENTIERE¶¬¬    NSIZ¬¬: ATTR_NBR;¬¬¬¬¬¬--| NOMBRE D ATTRIBUTS DU NOEUD (SI ENTETE) OU INDICATEUR D
     E COMPLEMENT 0+ 1- POUR ABSS¶¬¬  END CASE;¶¬¬END RECORD;¶¬¬FOR TREE'SIZE USE 32;¶¬¬FOR TREE USE RECORD¬AT MOD 4;¶¬¬¬PT¬AT 0 RANGE 0..1;¶¬¬¬LN¬AT 0 RAN
     GE 2..8;¶¬¬¬SLN¬AT 0 RANGE 2..8;¶¬¬¬NSIZ¬AT 0 RANGE 2..8;¶¬¬¬PG¬AT 0 RANGE 9..23;¶¬¬¬SPG¬AT 0 RANGE 9..23;¶¬¬¬ABSS¬AT 0 RANGE 9..23;¶¬¬¬COL¬AT 0 RANGE
0020  24..31;¶¬¬¬TY¬AT 0 RANGE 24..31;¶¬¬¬NOTY¬AT 0 RANGE 24..31;¶¬¬¬END RECORD;¶¶  TREE_NIL¬¬: CONSTANT TREE¬:= (P, TY¬=> DN_NIL,   PG => 0, LN => 0);¶¶
     TYPE SEQ_TYPE¬¬IS RECORD¶¬¬¬  FIRST, NEXT¬: TREE;¶¬¬¬END RECORD;¶¶¬--| ACCES¬A L'ARBRE¶¶  FUNCTION  MAKE¬¬( NN :NODE_NAME )¬¬¬¬RETURN TREE;¬--| AJOUTE
      UN NOEUD¬DE TYPE NODE_NAME¶¶  PROCEDURE D¬¬( AN :ATTRIBUTE_NAME; T :TREE; V :TREE );¬¬¬¬--| ECRITURE D'UN ATTRIBUT CONTENANT UN¬ARBRE¶  FUNCTION  D¬¬(
      AN :ATTRIBUTE_NAME; T :TREE¬)¬¬RETURN TREE;¬--| LECTURE D'UN ATTRIBUT CONTENANT UN ARBRE¶¶  PROCEDURE DB¬¬( AN :ATTRIBUTE_NAME; T :TREE; V :BOOLEAN )
     ;¬¬¬--| ECRITURE D'UN ATTRIBUT CONTENANT UN¬BOOLEEN¶  FUNCTION  DB¬¬( AN :ATTRIBUTE_NAME; T :TREE¬)¬¬RETURN BOOLEAN;¬--| LECTURE D'UN ATTRIBUT CONTENA
     NT UN BOOLEEN¶¶  PROCEDURE DI¬¬( AN :ATTRIBUTE_NAME; T :TREE; V :INTEGER );¬¬¬¬--| ECRITURE D'UN ATTRIBUT CONTENANT UN¬ENTIER (16 BITS)¶  FUNCTION  DI¬
     ¬( AN :ATTRIBUTE_NAME; T :TREE¬)¬¬RETURN INTEGER;¬--| LECTURE D'UN ATTRIBUT CONTENANT UN ENTIER (16¬BITS)¶¶  FUNCTION  LIST¬¬( T :TREE¬)¬¬¬¬RETURN SEQ
     _TYPE;¬--| REND LA LISTE CONTENUE DANS UN NOEUD POINTE PAR LE POINTEUR D'ARBRE¶  FUNCTION  IS_EMPTY¬( S :SEQ_TYPE )¬¬¬¬RETURN BOOLEAN;¬--| TESTE¬UNE L
     ISTE¶  PROCEDURE POP¬¬( S :IN OUT SEQ_TYPE; T :OUT TREE );¬¬¬¬--| EXTRAIT UN ELEMENT DE LISTE ET REPOINTE S SUR¬LE RESTE¶¶  FUNCTION  PRINT_NAME¬( T :
     TREE¬)¬¬¬¬RETURN STRING;¬--| TXTREP OR SYMBOL_REP¶  FUNCTION  NODE_IMAGE¬( NN :NODE_NAME )¬¬¬¬RETURN STRING;¬--| CHAINE REPRESENTANT UN NOEUD¶  FUNCTI
0030 ON  ATTR_IMAGE¬( AN :ATTRIBUTE_NAME )¬¬¬RETURN STRING;¬--| CHAIUNE REPRESENTANT UN NOM D'ATTRIBUT¶¶¬--| AFFICHAGE DE TOUT OU PARTIE DE L'ARBRE¶¶  --|-
     ----------------------------------------------------------------------------------------¶  --|¬¬PRINT_NOD¬¬¬¬¬¬¬¬--| POUR L'AFFICHAGE D'ELEMENTS
      D'ARBRE¬DIANA¶  --|-------------------------------------------------------------------------------------------¶  PACKAGE¬PRINT_NOD¬IS¶¶    PROCED
     URE PRINT_TREE¬( T :TREE¬);¶    FUNCTION  L_PRINT_TREE¬( T :TREE¬)¬¬RETURN NATURAL;¬¬¬--| RETOURNE LE NOMBRE DE CARACTERES¶    PROCEDURE PRINT_NODE¬(
     T :TREE; INDENT :NATURAL :=0 );¶¶  --|-----------------------------------------------------------------------------------------¶  END PRINT_NOD;
     ¶¶¶¶  PRAGMA INLINE ( DB );¶  PRAGMA INLINE ( DI );¶  PRAGMA INLINE ( D¬);¶¶--|-------------------------------------------------------------------------------
     -------------------------¶END IDL;¶
```